Digital Image Analysis Tutorial 13

```
"""
AI531C T13 — The Metrics Clinic
Run: python T13_studio.py
Modes: python T13_studio.py support
       python T13_studio.py extension
Own masks: python T13_studio.py --gt gt.png --pred prediction.png

Conventions:
- Non-zero pixels are foreground.
- Default demo: skimage coins, T02-style Gaussian + Otsu GT.
- Prediction: GT reduced to 1/8 resolution and enlarged with nearest-neighbour.
- Boundary band: dilation of the GT morphological-gradient boundary using
  a width x width square kernel. Boundary IoU is evaluated inside this band.
"""

import os
import sys
import argparse
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage import data

HERE = os.path.dirname(os.path.abspath(__file__))
INP = os.path.join(HERE, "inputs")
OUT = os.path.join(HERE, "expected_outputs")
os.makedirs(INP, exist_ok=True)
os.makedirs(OUT, exist_ok=True)

DEFAULT_BAND = 7
MIN_AREA = 200
```

```
def demo_pair():
    image = data.coins()
    cv2.imwrite(os.path.join(INP, "coins.png"), image)

    smooth = cv2.GaussianBlur(image, (5, 5), 1)
    _, gt = cv2.threshold(
        smooth, 0, 1, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    h, w = gt.shape
    coarse = cv2.resize(
        gt, (max(1, w // 8), max(1, h // 8)),
        interpolation=cv2.INTER_AREA
    )
    pred = cv2.resize(
        coarse, (w, h), interpolation=cv2.INTER_NEAREST
    )
    pred = (pred > 0).astype(np.uint8)

    cv2.imwrite(os.path.join(INP, "T13_demo_gt.png"), gt * 255)
    cv2.imwrite(os.path.join(INP, "T13_demo_prediction.png"), pred * 255)
    return gt.astype(np.uint8), pred

def load_mask(path):
    image = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError(f"Cannot read mask: {path}")
    return (image > 0).astype(np.uint8)

def metrics(gt, pred):
    g, p = gt.astype(bool), pred.astype(bool)
    tp = int(np.logical_and(g, p).sum())
    tn = int(np.logical_and(~g, ~p).sum())
```

```
    fp = int(np.logical_and(~g, p).sum())
    fn = int(np.logical_and(g, ~p).sum())
    total = tp + tn + fp + fn

    return {
        "TP": tp, "TN": tn, "FP": fp, "FN": fn,
        "Accuracy": (tp + tn) / total if total else 0,
        "Dice": 2 * tp / (2 * tp + fp + fn)
                if (2 * tp + fp + fn) else 1,
        "IoU": tp / (tp + fp + fn) if (tp + fp + fn) else 1,
        "Sensitivity": tp / (tp + fn) if (tp + fn) else 0
    }

def make_boundary_band(gt, width):
    boundary = cv2.morphologyEx(
        gt.astype(np.uint8), cv2.MORPH_GRADIENT,
        np.ones((3, 3), np.uint8)
    )
    return cv2.dilate(
        boundary, np.ones((width, width), np.uint8), iterations=1
    ).astype(bool)

def boundary_iou(gt, pred, width):
    band = make_boundary_band(gt, width)
    g = np.logical_and(gt.astype(bool), band)
    p = np.logical_and(pred.astype(bool), band)
    union = np.logical_or(g, p).sum()
    intersection = np.logical_and(g, p).sum()
    return intersection / union if union else 1.0

def support():
    print("[SUPPORT] Toy overlap calculation")
    print("    |A| = 150, |B| = 120, overlap = 90")
    dice = 2 * 90 / (150 + 120)
```

```
    iou = 90 / (150 + 120 - 90)
    print(f"    Dice = 2 x 90 / (150 + 120) = {dice:.3f}")
    print(f"    IoU  = 90 / (150 + 120 - 90) = {iou:.3f}")
    print(f"    Identity check: 2 x IoU / (1 + IoU) = {2*iou/(1+iou):.3f}")


def extension(gt, pred):
    n, labels, stats, _ = cv2.connectedComponentsWithStats(
        gt.astype(np.uint8), connectivity=8
    )
    rows = []
    for i in range(1, n):
        area = int(stats[i, cv2.CC_STAT_AREA])
        if area < MIN_AREA:
            continue
        obj = labels == i
        pred_area = int(np.logical_and(pred.astype(bool), obj).sum())
        score = 2 * pred_area / (area + pred_area) if area + pred_area else 1
        rows.append((i, area, score))

    rows.sort(key=lambda x: x[2])
    print("[EXTENSION] Per-object Dice")
    for i, area, score in rows:
        print(f"    object #{i}: area={area}, Dice={score:.3f}")
    if rows:
        values = [r[2] for r in rows]
        print(f"    worst={min(values):.3f}, mean={np.mean(values):.3f}")
        print("    Inspect the lowest-scoring object patches to identify")
        print("    missed regions or incorrect foreground predictions.")


def main():
    parser = argparse.ArgumentParser(description="AI531C T13 Metrics Clinic")
    parser.add_argument("--gt", help="Ground-truth binary mask path")
    parser.add_argument("--pred", help="Predicted binary mask path")
    parser.add_argument("--band", type=int, default=DEFAULT_BAND)
```

```
    args = parser.parse_args()

    if args.band < 1:
        parser.error("--band must be >= 1")
    if bool(args.gt) != bool(args.pred):
        parser.error("Provide both --gt and --pred, or neither.")

    if args.gt:
        gt, pred = load_mask(args.gt), load_mask(args.pred)
    else:
        gt, pred = demo_pair()

    if gt.shape != pred.shape:
        raise ValueError(f"Mask sizes differ: GT={gt.shape},
prediction={pred.shape}")

    print("AI531C T13 — The Metrics Clinic")
    print(f"Image size: {gt.shape[1]} x {gt.shape[0]}")
    print("Convention: non-zero = foreground; masks must be aligned.")
    print(f"Boundary convention: GT boundary band, width={args.band} px.")

    m = metrics(gt, pred)
    print("\\n[1] Confusion counts")
    print(f"    TP={m['TP']} TN={m['TN']} FP={m['FP']} FN={m['FN']}")

    print("\\n[2] Segmentation metrics")
    for key in ("Accuracy", "Dice", "IoU", "Sensitivity"):
        print(f"    {key:<14}: {m[key]:.3f}")

    bscore = boundary_iou(gt, pred, args.band)
    print(f"    {'Boundary IoU':<14}: {bscore:.3f}")

    print("\\n[3] Boundary-band width sweep")
    widths = [3, 7, 15]
```

```
    scores = []
    for width in widths:
        score = boundary_iou(gt, pred, width)
        scores.append(score)
        print(f"    width {width:2d} px -> {score:.3f}")

    all_fg = np.ones_like(gt, dtype=np.uint8)
    baseline = metrics(gt, all_fg)["Dice"]
    print("\\n[4] All-foreground baseline")
    print(f"    GT foreground fraction: {gt.mean():.3f}")
    print(f"    All-foreground Dice: {baseline:.3f}")

    band = make_boundary_band(gt, args.band)
    fig, ax = plt.subplots(1, 4, figsize=(14, 4))
    ax[0].imshow(gt, cmap="gray")
    ax[0].set_title("Ground truth")
    ax[1].imshow(pred, cmap="gray")
    ax[1].set_title("Prediction")
    ax[2].imshow(band, cmap="gray")
    ax[2].set_title(f"GT boundary band ({args.band}px)")
    ax[3].plot(widths, scores, "o-")
    ax[3].set_xlabel("Band width (px)")
    ax[3].set_ylabel("Boundary IoU")
    ax[3].set_title("Width sensitivity")
    for a in ax[:3]:
        a.axis("off")
    ax[3].grid(alpha=0.25)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT, "T13_panel.png"), dpi=130)
    plt.close()
    print("\\nsaved expected_outputs/T13_panel.png")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "support":
```

```
    support()
elif len(sys.argv) > 1 and sys.argv[1] == "extension":
    gt, pred = demo_pair()
    extension(gt, pred)
else:
    main()
```

Input:



Output:

